



Programming Languages

Departamento de Engenharia Informática, Técnico Lisboa

MEIC P4 25.26

Project Statement (Phase 1)

© Luís Caires (luís.caires@tecnico.ulisboa.pt | <https://luiscaires.org>)

Objective of Project Phase 1

Implement an interpreter for the Rusty v1 functional-imperative language

- Rusty v1 is essentially the R1 functional-imperative language studied in the course lectures (M2 and M3), but with some extensions.
- Base your implementation in the big-step environment-based semantics studied in the course, and exploit the basic interpreter for R0 developed in the practical lab sessions L2 and L3.
- Implement all constructs defined in this statement, in particular a new data type for **immutable tuple structs** (inspired from Rust).
- Implement safe recursive definitions (this is a special challenge t)
- Implement **full dynamic type-checking**, ensuring that all undefined operations are correctly signalled using exceptions and adequate error messages.
- Write examples, illustrating the use of Rusty 1.
- The next slides describe the details on the Rusty v1 language, and on the requirements of the Project.

Rusty v1 language (abstract syntax)

EE -> **id** | **num** | **true** | **false** | **string** | **()**
| EE **+** EE | EE **-** EE
| EE ***** EE | EE **/** EE | **-**EE | **(** EE **)**
| EE **==** EE | EE **>** EE | EE **>=** EE | **...**
| EE **&&** EE | EE **||** EE | **~** EE
| **tag**(EE,EE,...)
| **match** EE { **tag**(**id**,**id**,...) => EE, **else** => EE }
| **let** (**id** = EE)+ **in** EE
| **cell** (EE) | EE **:=** EE | **!**EE
| **if** EE **then** EE **else** EE
| **while** EE **do** EE
| **print!** EE | **println!** EE | EE **;** EE
| **abs** (**id**, EE)
| **app** (EE, EE)

We provide a **initial parser** in the shared drive that covers most of the Rusty v1 Language, you may need to add or modify parts of the grammar, to generate the AST. We provide the definition of **all tokens**.

Notes:

() denotes the “unit” value (cf. Python **None**)

The only operator for string is **+** which is overloaded with integer addition. Informally, the value of E1 + E2 is the concatenation of strings. If one E1 or E2 evaluates to a string, the value of the other is converted to a string and the result is the concatenation. Otherwise the values of E1 and E2 must be integers.

Rusty v1 language (JavaCC concrete syntax)

See File **ParserRustyV1.jj !**

```
Start ::= <EOF>
        | Let <TERM>
Let ::= ( Seq | ( ( <LET> <Id> <EQUAL> BA <SEMIC> )+ Seq ) )
Seq ::= SeqExp ( ( <SEMIC> ) SeqExp )*
SeqExp ::= BA ( ( <ASSIGN> ) BA )*
BA ::= BM ( ( <OR> ) BM )*
BM ::= Rel ( ( <AND> ) Rel )*
Rel ::= Exp ( ( <EQ> | <GT> | <LT> | <GE> | <LE> | <NEQ> ) Exp )?
Exp ::= Term ( ( <PLUS> | <MINUS> ) Term )*
Term ::= Fact ( <MUL> Fact | <DIV> Fact | <LPAR> ( ( Let ( ( <RPAR> ) | ( (
    <COMMA> Let )+ <RPAR> ) ) ) | <NONE> ) *
StructTuple ::= <TAG> ( <LPAR> Let ( <COMMA> Let ) * <RPAR> | <NONE> )
Function ::= ( <BAR> <Id> ( <COMMA> <Id> ) * <BAR> Fact | <OR> Fact )
Pattern ::= <TAG> ( <LPAR> <Id> ( <COMMA> <Id> ) * <RPAR> | <NONE> )
Match ::= <MATCH> Let <LBRA> Pattern <ARROW> Let <ELSE> <ARROW>
    Let <RBRA>
Fact ::= ( <Num> | <TRUE> | <FALSE> | <NONE> | <Id> | <MINUS> Fact |
    <NOT> Fact | Function | <IF> BA <LBRA> Let <RBRA> <ELSE>
    <LBRA> Let <RBRA> | <LPAR> Let <RPAR> | <LBRA> Let <RBRA> |
    <PRINT> Fact | <PRINTLN> Fact | <WHILE> BA <LBRA> Let
    <RBRA> | <CELL> Fact | <DEREF> Fact | StructTuple | Match )
```

Rusty v1 language (JavaCC tokens)

See File **ParserRustyV1.jj** !

```
<DEFAULT> TOKEN : {  
<LET: "let">  
  <TRUE: "true">  
  <FALSE: "false">  
  <PLUS: "+">  
  <MINUS: "-">  
  <MUL: "*">  
  <DIV: "/">  
  <LPAR: "(">  
  <RPAR: ")">  
  <LBRA: "{">  
  <RBRA: "}">  
  <EQUAL: "=">  
  <COLON: ":">  
  <SEMIC: ";">  
  <TERM: ";;">  
  <COMMA: ",">  
  <AND: "&&">  
  <BAR: "|">  
  <OR: "||">  
  <EQ: "==">  
  <GT: ">">  
  <GE: ">=">
```

```
  <LT: "<">  
  <LE: "<=">  
  <NEQ: "~=">  
  <NOT: "~">  
  <NONE: "()">  
  <ASSIGN: "==">  
  <CELL: "cell">  
  <DEREF: "!">  
  <MATCH: "match">  
  <IF: "if">  
  <ELSE: "else">  
  <WHILE: "while">  
  <PRINT: "print!">  
  <PRINTLN: "println!">  
  <ARROW: "=>">  
  <TAG: "#" ["a"-"z", "A"-"Z"] (["a"-"z", "A"-"Z", "0"-"9"])*>  
  <String: "\"" (~["\"", "\\"] | "\\\" ["n", "t", "b", "r", "f", "\\\"", "\\"])* "\">  
  <Id: ["a"-"z", "A"-"Z"] (["a"-"z", "A"-"Z", "0"-"9"])*>  
  <Num: (["0"-"9"])+>  
}
```

Rusty v1 language (semantics)

Rusty v1 is an extension of the functional imperative-language R1 studied in the course, extended with **strings** and **struct tuples**.

A struct tuple is similar to a struct tuple in Rust, an immutable named tuple.

Struct tuples are constructed by listing a sequence of values under a tag, like in
`#Color(9,2,3)`

or

`#Person("luis", 55)`

A tag is like an identifier, but must start by the special character #.

Rusty v1 language (semantics)

Struct tuples may be decomposed (deconstructed) using the match construct, for instance

```
match EE {  
    #Color(r,g,b) => r*r+g*g+b*b,  
    else => -1  
}
```

This expression evaluates to $V_1 * V_1 + V_2 * V_2 + V_3 * V_3$ if EE expression evaluates to a struct tuple of the form #Color(V_1, V_2, V_3). Otherwise, it evaluates to -1.

Below we specify the syntax and semantics of the primitives related to struct tuples.

Rusty v1 language (program expressions)

Rusty v1 is an extension of the functional imperative-language R1 studied in the course, extended with strings and **struct tuples**.

The term expressions of Rusty v1 are the same as R1, with the two constructions.

M, N (*Terms*) ::=

| ... (basic terms of LI)

| $\#tag(N_1, \dots, N_n)$ (*struct tuple constructor*)

| $\text{match } M \{ \#tag(x_1, \dots, x_n) \Rightarrow N, \text{ else } N' \}$ (*struct tuple destructor*)

Rusty v1 language (values)

Rusty v1 is an extension of the functional imperative-language R1 studied in the course, extended with strings and **struct tuples**.

We describe here the values of Rusty v1.

$\frac{\text{(integer)}}{\text{value } n}$	$\frac{\text{(string)}}{\text{value } str}$	$\frac{}{\text{value false}}$	$\frac{}{\text{value true}}$	$\frac{\text{(reference)}}{\text{value } \ell}$
	$\frac{}{\text{value clos}(\mathcal{E}, \lambda x . M)}$	$\frac{\text{value } V_1 \ \dots \ \text{value } V_n}{\text{value } \#tag(V_1, \dots, V_n)}$		

Rusty v1 language (Semantics)

Rusty v1 is an extension of the functional imperative-language R1 studied in the course, extended with **strings** and **struct tuples**.

Here we specify the big step operational semantics of Rusty v1 (only the new rules)

$\mathcal{E}; \mathcal{S}; M \Downarrow N; \mathcal{S}'$ (M evaluates to N in environment $\mathcal{E}$ and store $\mathcal{S}$)

$$\frac{\mathcal{E}; \mathcal{S}; M_1 \Downarrow V_1; \mathcal{S}_1 \quad \dots \quad \mathcal{E}; \mathcal{S}_{n-1}; M_n \Downarrow V_n; \mathcal{S}_n}{\mathcal{E}; \#tag(M_1, \dots, M_n) \Downarrow \#tag(V_1, \dots, V_n); \mathcal{S}'}$$

$$\frac{\mathcal{E}; \mathcal{S}; M \Downarrow V; \mathcal{S}' \quad V \neq \#tag(V_1, \dots, V_n) \quad \mathcal{E}; \mathcal{S}'; N' \Downarrow U; \mathcal{S}''}{\mathcal{E}; \mathcal{S}; \text{match } M \{ \#tag(x_1, \dots, x_n) \Rightarrow N, \text{ else } N' \} \Downarrow U; \mathcal{S}''}$$

$$\frac{\mathcal{E}; \mathcal{S}; M \Downarrow \#tag(V_1, \dots, V_n); \mathcal{S}' \quad \mathcal{E}[x_1 \rightarrow V_1] \dots [x_n \rightarrow V_n]; \mathcal{S}'; N \Downarrow U; \mathcal{S}''}{\mathcal{E}; \mathcal{S}; \text{match } M \{ \#tag(x_1, \dots, x_n) \Rightarrow N, \text{ else } \Rightarrow N' \} \Downarrow U; \mathcal{S}''}$$

match construct (examples)

```
match #Cons(2+1,2>0) {  
  #Cons(x,y) => x  
  else => -1  
};;
```

evaluates to 3

```
match #Nil() {  
  #Nil() => 2+2  
  else => false  
};;
```

evaluates to 4

```
match #Foo(2) {  
  #Nil() => 2+2  
  else => 2<0  
};;
```

```
let sumlist = |l| match l {  
  #Cons(v,r) => v + sumlist (r)  
  else => match l {  
    #Nil() => 0  
    else => ()  
  }  
};
```

```
let l0 = #Cons(1,#Cons(2,#Cons(3,#Cons(4,#Nil()))));  
sumlist (l0);;
```

evaluates to 10

(Challenge - worth 5%)

Recursive definitions like

```
let fact = |n| { if (n=0) then 1 else n*fact(n-1) };  
fact(6);
```

are easy to implement if your interpreter evaluates the initialisation expressions of a let block in the **new environment** instead of the initial environment. This works well in “recursive” references appear in the initialisation expressions behind a lambda binder.

For instance, using the method above, the program will loop forever before even starting to evaluate the body of the let expression!

```
let x = x+1  
2*x;
```

(Challenge - worth 5%)

As part of the project we may solve the following special challenge:

We say that a `let x1 = E1; let x2=E2; ....; Body` is “recursion unsafe” if some identifier `xi` is directly accessible from itself via the expressions. An identifier is accessible in an expression `E` if only appears free in `E` behind a function abstraction.

For example, the first program in the last slide is recursion safe, but the second is not.

Implement in your Rusty v1 interpreter a “safe recursion” static check, to ensure that the interpreter will reject any “recursion unsafe” unsafe let block with a error message. the check must be done after parsing a let block, and before evaluation a program.

TO-DO List (summary)

- 0 - Your starting point is the **R0** interpreter already developed in the lab classes. The goal of those classes was precisely to offer you a smooth “onboarding” on the project.
- 1 - You have to implement the required AST nodes types for all constructs, to represent the abstract syntax tree of any Rusty v1 program as defined in this statement.
- 2 - You also have to define the new required values types, namely VCell (for memory references), VString (for string values), and VStructTuple (for struct tuples).
- 3 - (Challenge) you may implement the “safe recursion” checks on let block (see last slide). The challenge will be graded 1 point on 20.
- 4 - You may also submit some sample code of your own authorship, to “demonstrate” the interpreter. You may write up to 5 small examples, but please be creative. You may include programs that do not have runtime errors, and programs that have runtime errors, commenting them as “/* KO */”.

TO-DO List (summary)

Level 0 - implement the basic functional language (14/20)

Level 1 = Level 0 + imperative fragment (16 / 20)

Level 2 = Level 1 + struct tuples (18 / 20)

Examples = 1 / 20

Safe Recursion challenge 1

Submission Instructions

1 - Place your code in a gitlab repo shared with me
(luis.caires@tecnico.ulisboa.pt)

NOTE: the contents of the shared repo **must not be changed** after the deadline.

3- You may use the software available in the gitlab RNL standard installation and javacc (installation requested).

4 - Include a top level script “**makeit**”, that I may use to compile all your source code by typing “\.**makeit**” at the command line.

Appendix / Notes

- 1 - Some startup source code (Java) will be placed in the PL Course Drive. This startup code uses the javacc parser generator, which is quite simple to use.
- 2 - This project statement will be presented in the lab sessions, and the startup source code as well. The startup code only covers a very small fragment of the class, but already hints to the general structure of the interpreter we look for.
- 3 - You are free to pick another implementation language (C, C++ or Rust), and some other standard parser generator, if you wish, but that will take you lots of extra time to redo the parser, and is definitely **not advisable**.
- 4 - Some tests for the various fragments of the language will also be published by May 5th (rustytestsv1.md)
- 5 - For any question, please interact via the **course Slack**, or DM me via e-mail.